

Wakacyjne Wyzwanie

Z  KN Solvro

Jak i gdzie przechowywać dane?

Krystian Śliwiński
17.08.2026



KN Solvro

Rozkład Jazdy

1. Wstęp
2. Czym są bazy danych?
3. Jakie są ich rodzaje?
4. SQL
5. Modelowanie baz danych
6. DBMS
7. ORM
8. Prisma



KN Solvro

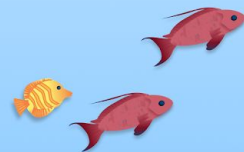
IT

Information Technology

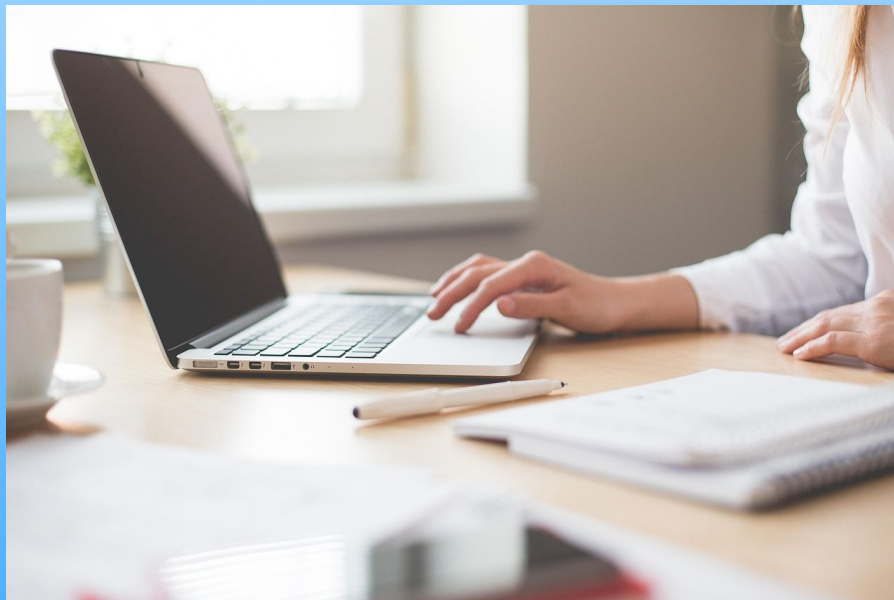
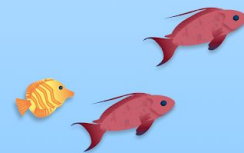


KN Solvro

Wstęp



Wstęp



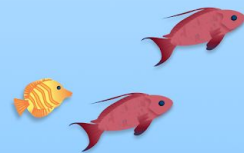
KN Solvro

Bazy danych



KN Solvro

Bazy danych

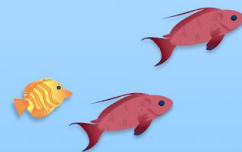


Baza danych – zbiór danych zapisanych z określonymi regułami. W węższym znaczeniu obejmuje dane cyfrowe gromadzone zgodnie z zasadami przyjętymi dla danego programu komputerowego specjalizowanego w gromadzeniu i przetwarzaniu tych danych.



KN Solvro

Bazy danych

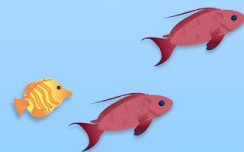


```
Podaj swoje imię: Marek
Podaj ocenę (lub wpisz "koniec" aby zakończyć): 3
Podaj ocenę (lub wpisz "koniec" aby zakończyć): 5
Podaj ocenę (lub wpisz "koniec" aby zakończyć): 2
Podaj ocenę (lub wpisz "koniec" aby zakończyć): 1
Podaj ocenę (lub wpisz "koniec" aby zakończyć): koniec
```

```
Podaj swoje imię: Nie-Marek
Podaj ocenę (lub wpisz "koniec" aby zakończyć): 1
Podaj ocenę (lub wpisz "koniec" aby zakończyć): 1
Podaj ocenę (lub wpisz "koniec" aby zakończyć): 1
Podaj ocenę (lub wpisz "koniec" aby zakończyć): koniec
```



Bazy danych



- W jaki sposób odróżnimy Janka Kowalskiego z IVa z Jankiem Kowalskim z IIIb?
- Jak zapiszemy te oceny w pliku?
- Jak będziemy wyszukiwać uczniów?

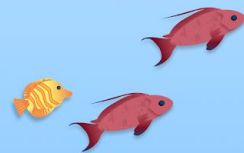


Rodzaje baz danych

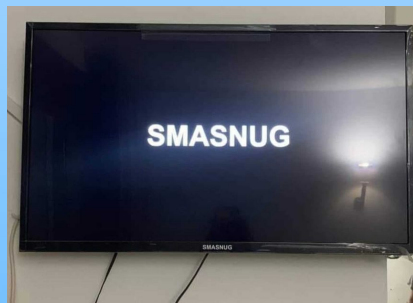


KN Solvro

Bazy typu klucz-wartość



Bazy dokumentowe



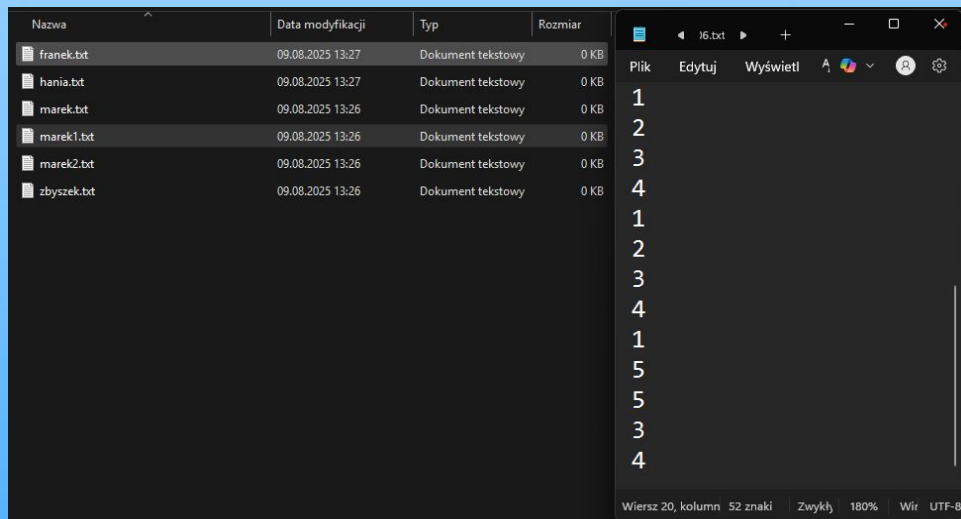
Rozdzielczość:



Hamulce:
Szimanochy



Bazy danych



Relacyjne bazy danych



KN Solvro

Relacyjne bazy danych

Uczen	
PK	<u>id</u>
	imie
	nazwisko
	id_klasy

Klasa	
PK	<u>id</u>
	rok
	symbol

```
CREATE TABLE Uczen (  
  id INT PRIMARY KEY,  
  imie VARCHAR(50),  
  nazwisko VARCHAR(50),  
  id_klasy INT,  
  FOREIGN KEY(id_klasy) REFERENCES Klasa(id)  
);
```

```
CREATE TABLE Klasa (  
  id INT PRIMARY KEY,  
  rok INT,  
  symbol CHAR(1)  
);
```



SQL



KN Solvro

CRUD

Create, Read, Update, Delete



KN Solvro

Create



```
INSERT INTO Uczen (id, imie, nazwisko, id_klasy)
VALUES (1, 'Janek', 'Kowalski', 4);
```

Read

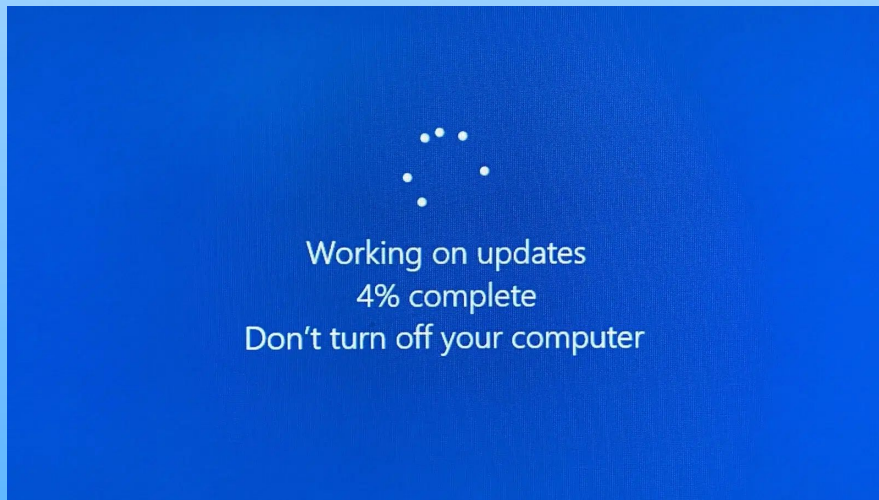
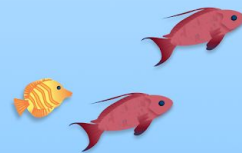


```
SELECT imie, nazwisko FROM Uczen WHERE id_klasy = 4;
```



KN Solvro

Update



```
UPDATE Ucen SET id_klasy = 5 WHERE id = 1;
```



KN Solvro

Delete



```
DELETE FROM Ucen WHERE id = 1;
```



KN Solvro

SQL



SQL CHEATSHEET

by @codes.learning

Commands/ Clauses

SELECT Select data from database
FROM Specify table where data from
WHERE Filter query to match a condition
AS Rename column or table with alias
JOIN Combine rows from 2 or more tables
AND Combine query conditions. All must be met
OR Combine query conditions. One must be met
LIKE Find rows returned. Use also **NOT** and **TOP**
IN Specify multiple values when using **WHERE**
EXISTS Return true or a specified condition
ALL Return only rows with a **NULL** value
GROUP Search for patterns in columns
ORDER Order transaction to database
ROLLBACK Undo a transaction from
ALTER TABLE Add/Remove columns from table
CREATE Create new table
CREATE TABLE, DATABASE, INDEX or VIEW
DELETE Delete rows from table
INSERT Add single row to table
UPDATE Update **TABLE**, **COLUMN**, or **INDEX**
GROUP BY Group data into logical sets
ORDER BY Set order of result. Use **DESC** to reverse order
UNION Same as **WHERE** but from groups
COUNT Count number of rows
SUM Return sum of column
AVG Return average of column
MIN Return min value of column
MAX Return max value of column

Joins



= INNER JOIN



= LEFT JOIN



= RIGHT JOIN



= FULL OUTER JOIN

Examples

Select all columns with filter applied
SELECT * FROM **tbl**
WHERE col > 5
 Select first 10 rows for two columns
SELECT col1, col2
 FROM **tbl** LIMIT 10;
 Select all columns with multiple filters
SELECT * FROM **tbl**
WHERE col > 1 OR col2 < 2;
 Select all rows from **col1** & **col2** ordering by **col1**
SELECT col1, col2
 FROM **tbl** ORDER BY 1;
 Return count of rows in table
SELECT COUNT(*)
 FROM **tbl**;
 Return sum of **col1**
SELECT SUM(col1)
 FROM **tbl**;
 Return max value for **col1**
SELECT MAX(col1)
 FROM **tbl**;
 Compute summary stats by grouping **col2**
SELECT AVG(col1) FROM **tbl GROUP BY col2;**
 Combine data from 2 tables using left join
SELECT * FROM **tbl1 AS **tbl1** LEFT JOIN **tbl2** AS **tbl2** ON (**tbl1.col1** = **tbl2.col1**);**
 Aggregate and filter result
SELECT **col1, COUNT(*) AS total**
 FROM **tbl** GROUP BY **col1** HAVING COUNT(*) > 10;
 Implementation of CASE statement
SELECT **col1, CASE WHEN **col1** < 10 THEN Rows less than 10' WHEN **col1** < 10 THEN Rows less than 10' ELSE 100' END AS NewcolumnName**
 FROM **tbl**;

Data Definition Language

CREATE
CREATE DATABASE **MyDatabase**;
CREATE TABLE **MYTable** (
id **INT**,
name **VARCHAR**(100) **NOT NULL**,
CREATE INDEX **MyIndexName** (
tbl **tablename** (**col**);

ALTER

ALTER TABLE **MyTable** (
DROP COLUMN **col**;
ALTER TABLE **MyTable** **ADD** **col** **INT**;
DROP
GROUP DATABASE **MyDatabase**;
DROP TABLE **MyTable**;

Data Manipulation Language

UPDATE
UPDATE **MyTable** SET **col1** = 10
WHERE col1 = 'something';
DELETE
DELETE FROM **MyTable** **WHERE col1 = 'something'**;
INSERT
INSERT INTO **MyTable** (**col1**, **col2**) **VALUES** (**val1**, **val2**);
SELECT
SELECT **col1**, **col2** FROM **MyTable**;

Order of Execution

- 1 FROM
- 2 WHERE
- 3 GROUP BY
- 4 HAVING
- 5 SELECT
- 6 ORDER BY
- 7 LIMIT



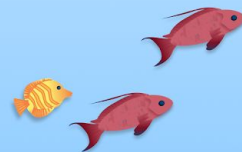
Learn to code
 @codes.learning

GET FREE CERTIFICATION COURSES &
 KNOWLEDGE JOIN OUR TELEGRAM CHANNEL
[LINK IN BIO](#)



KN Solvro

Typy danych



Category	Data Type	Description	Example Use
Numeric	INTEGER , BIGINT	Whole numbers of varying sizes	Unique IDs, counts
	NUMERIC , DECIMAL	Fixed-point numbers with precision	Balances, financial data
Character	VARCHAR , TEXT	Variable-length text	Names, descriptions
Date/Time	DATE , TIMESTAMP	Dates and timestamps	Event dates, logs
Boolean	BOOLEAN	True/False values	Status flags
Array	ARRAY	List of values of the same type	List of tags, multiple emails
Special	UUID , JSON	Unique identifiers, JSON data	User IDs, product attributes
Geometric	POINT , LINE	Geometric data types for coordinates	Location data



Modelowanie baz danych



KN Solvro

Encja



Element świata rzeczywistego, który można opisać przy pomocy atrybutów



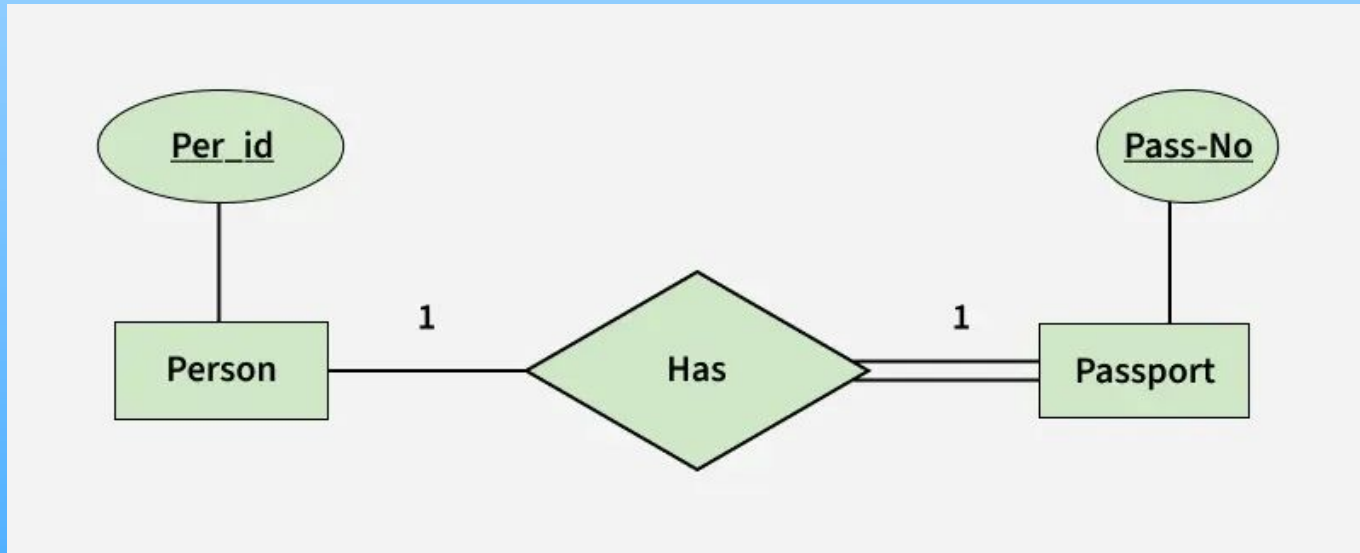
KN Solvro

Relacje między encjami



KN Solvro

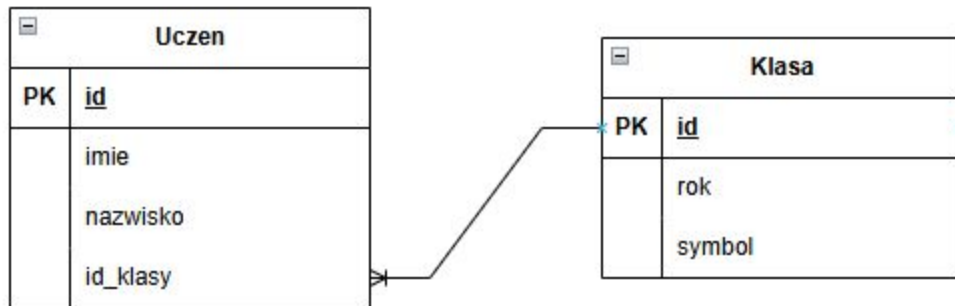
Relacja 1:1



Relacja 1:N



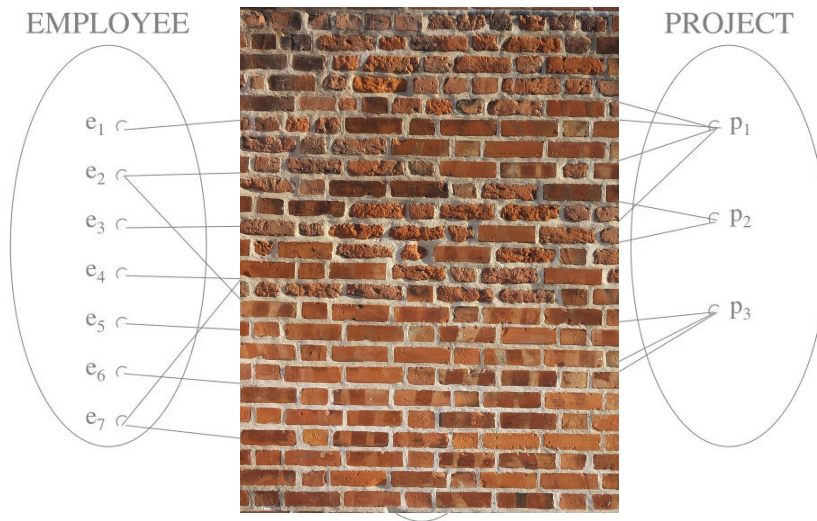
Relacja 1:N



KN Solvro

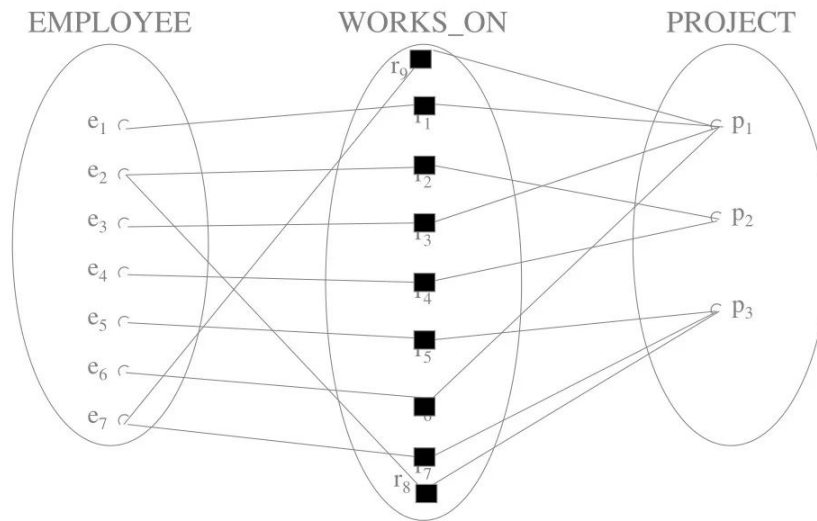
Relacja M:N

Many-to-many (M:N) RELATIONSHIP



Relacja M:N

Many-to-many (M:N) RELATIONSHIP



28



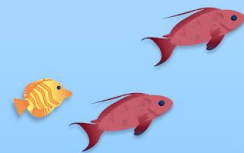
KN Solvro

Diagramy ERD



KN Solvro

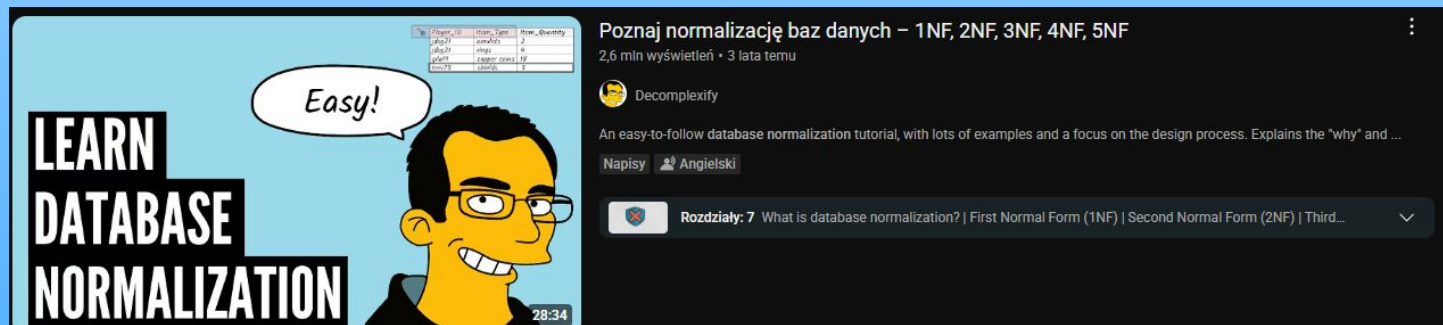
◦ Zróbmy sobie diagram...



KN Solvro

Normalizacja

https://www.youtube.com/watch?v=GFQaEYEc8_8&t



LEARN DATABASE NORMALIZATION

Easy!

Player ID	Item Type	Item Quantity
1	small	2
2	big	5
3	small	10
4	small	1

Poznaj normalizację baz danych – 1NF, 2NF, 3NF, 4NF, 5NF

2,6 mln wyświetleń • 3 lata temu

 Decomplexify

An easy-to-follow database normalization tutorial, with lots of examples and a focus on the design process. Explains the "why" and ...

Napisy  Angielski

Rozdziały: 7 What is database normalization? | First Normal Form (1NF) | Second Normal Form (2NF) | Third...



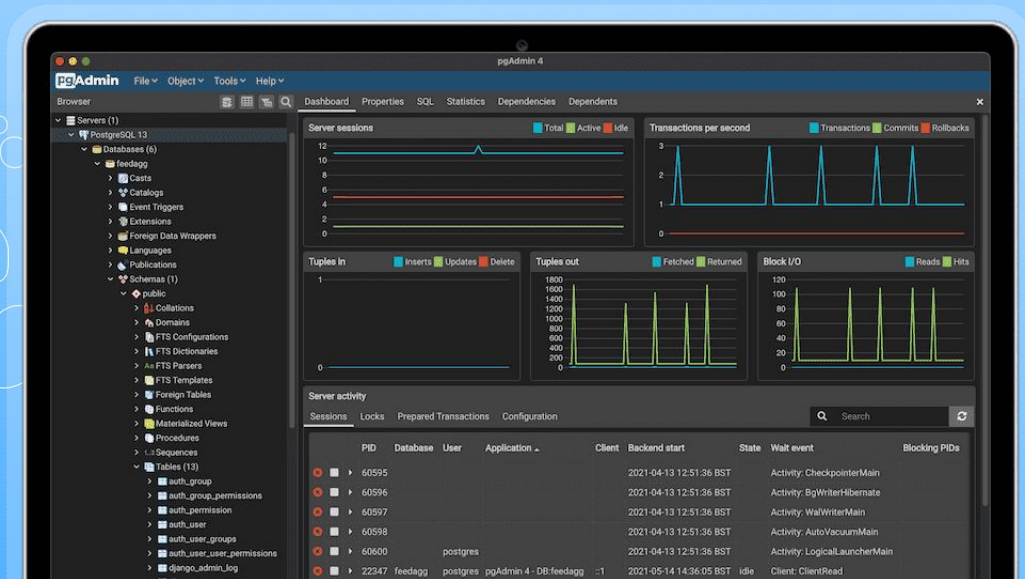
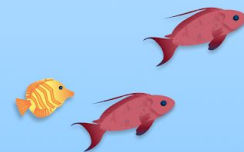
KN Solvro

DBMS



KN Solvro

Narzędzia do pracy z bazą danych



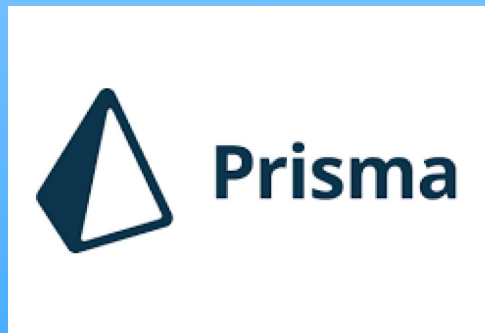
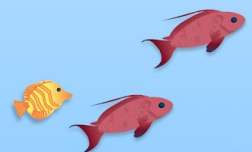
KN Solvro

ORM

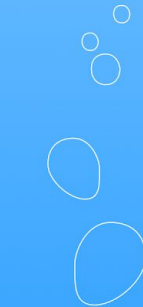
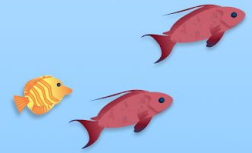
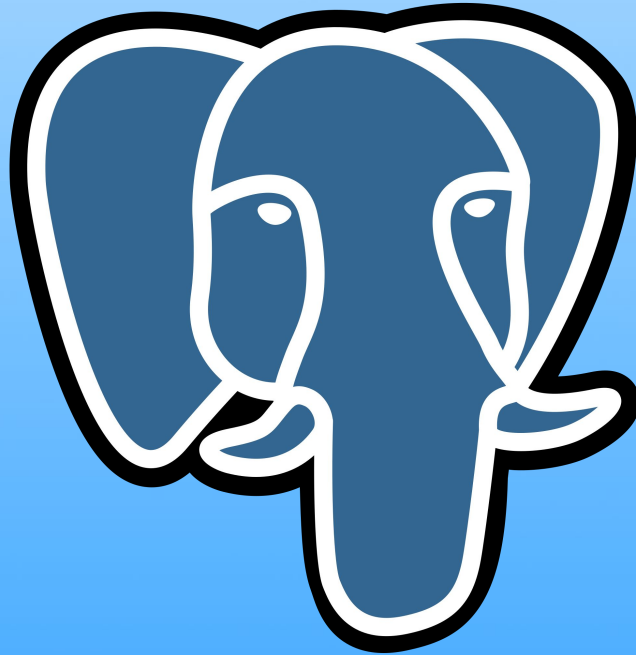


KN Solvro

ORM



◦ Czego potrzebujemy?



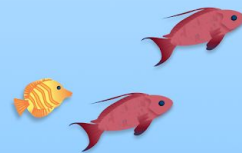
KN Solvro

Prisma



KN Solvro

Instalacja i konfiguracja Prisma



2. Install required dependencies

Install the packages needed for this quickstart:

bun pnpm yarn npm

```
npm install prisma @types/pg --save-dev
```

```
npm install @prisma/client @prisma/adapters pg dotenv
```

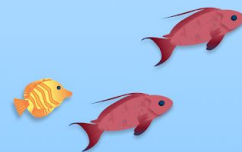


[https://www.prisma.io/docs/prisma-orm/
quickstart/prisma-postgres](https://www.prisma.io/docs/prisma-orm/quickstart/prisma-postgres)



KN Solvro

Instalacja i konfiguracja Prisma



4. Initialize Prisma ORM

You can now invoke the Prisma CLI by prefixing it with `npx`:

```
bun pnpm yarn npm
```

```
npx prisma
```

Next, set up your Prisma ORM project by creating your Prisma Schema file with the following command:

```
bun pnpm yarn npm
```

```
npx prisma init --output ../generated/prisma
```

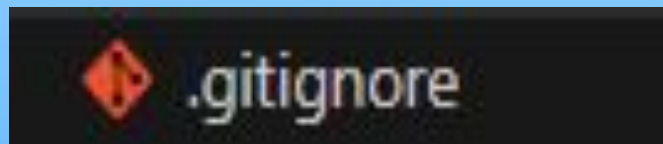
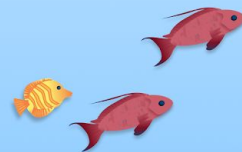
`prisma init` creates the Prisma scaffolding and a local `DATABASE_URL`.

[https://www.prisma.io/docs/prisma-orm/
quickstart/prisma-postgres](https://www.prisma.io/docs/prisma-orm/quickstart/prisma-postgres)



KN Solvro

Instalacja i konfiguracja Prisma



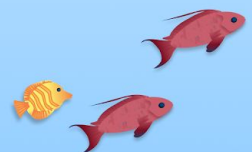
```
11  
12 DATABASE_URL="postgres://postgres:postgres@127.0.0.1:5432/nazwa_bazy"
```

[https://www.prisma.io/docs/prisma-orm/
quickstart/prisma-postgres](https://www.prisma.io/docs/prisma-orm/quickstart/prisma-postgres)



KN Solvro

Modele



5. Define your data model

Open `prisma/schema.prisma` and add the following models:

```
prisma/schema.prisma

generator client {
  provider = "prisma-client"
  output   = "../generated/prisma"
}

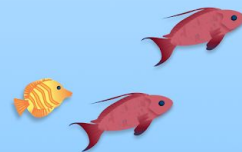
datasource db {
  provider = "postgresql"
}

+model User {
+  id    Int    @id @default(autoincrement())
+  email String @unique
+  name  String?
+  posts Post[]
+}

+model Post {
+  id        Int    @id @default(autoincrement())
+  title     String
+  content   String?
+  published Boolean @default(false)
+  author    User    @relation(fields: [authorId], references: [id])
+  authorId  Int
+}
```



Migracje

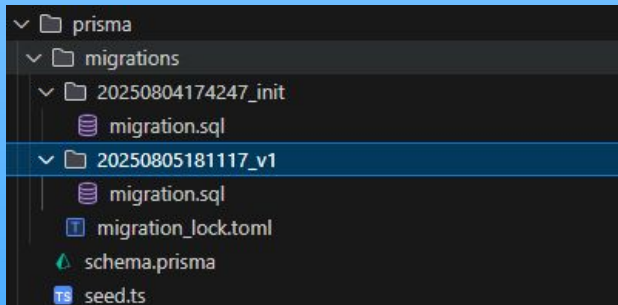


6. Create and apply your first migration

Create your first migration to set up the database tables:

```
bun pnpm yarn npm
```

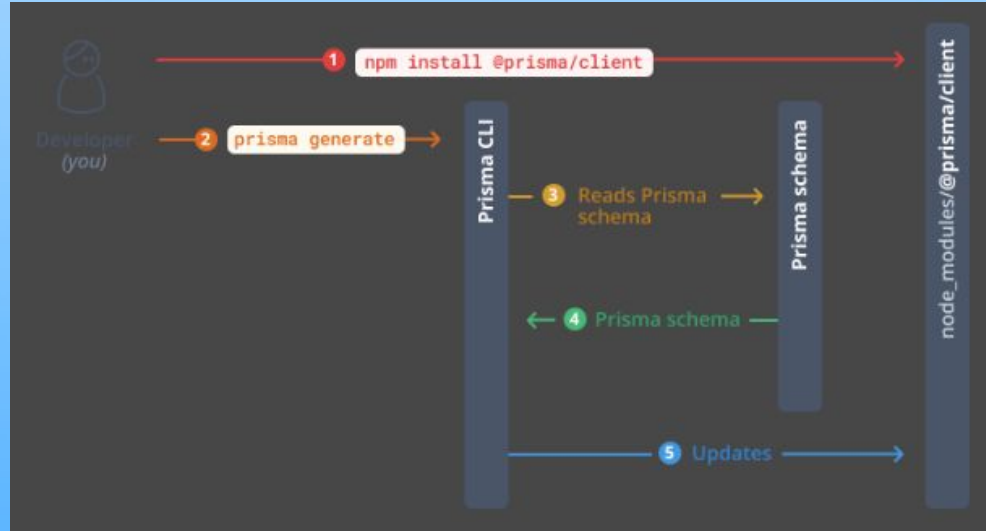
```
npx prisma migrate dev --name init
```



KN Solvro



Prisma Client



Now run the following command to generate the Prisma Client:

```
bun pnpm yarn npm
```

```
npx prisma generate
```



KN Solvro

Seedery

```
import { PrismaClient } from '@prisma/client'
const prisma = new PrismaClient()
async function main() {
  const alice = await prisma.user.upsert({
    where: { email: 'alice@prisma.io' },
    update: {},
    create: {
      email: 'alice@prisma.io',
      name: 'Alice',
      posts: {
        create: {
          title: 'Check out Prisma with Next.js',
          content: 'https://www.prisma.io/nextjs',
          published: true,
        },
      },
    },
  })
  const bob = await prisma.user.upsert({
    where: { email: 'bob@prisma.io' },
    update: {},
    create: {
      email: 'bob@prisma.io',
      name: 'Bob',
      posts: {
        create: [
          {
            title: 'Follow Prisma on Twitter',
            content: 'https://twitter.com/prisma',
            published: true,
          },
          {
            title: 'Follow Nexus on Twitter',
            content: 'https://twitter.com/nexusgql',
            published: true,
          },
        ],
      },
    },
  })
  console.log({ alice, bob })
}
```

Database service

```
npx nest generate module database  
npx nest generate service database
```

```
@Global()  
@Module({  
  providers: [PrismaService],  
  exports: [PrismaService],  
})  
export class PrismaModule {}
```

3. Create a Prisma service

You're now able to send database queries with Prisma Client. When setting up your NestJS application, you'll want to abstract away the Prisma Client API for database queries within a service. To get started, you can create a new `PrismaService` that takes care of instantiating `PrismaClient` and connecting to your database.

Inside the `src` directory, create a new file called `prisma.service.ts` and add the following code to it:

```
src/prisma.service.ts  
  
import { Injectable } from "@nestjs/common";  
import { PrismaClient } from "../generated/prisma/client.js";  
import { PrismaPg } from "@prisma/adapters-pg";  
  
@Injectable()  
export class PrismaService extends PrismaClient {  
  constructor() {  
    const adapter = new PrismaPg({  
      connectionString: process.env.DATABASE_URL as string,  
    });  
    super({ adapter });  
  }  
}
```

<https://www.prisma.io/docs/guides/frameworks/nestjs>



KN Solvro

Praca domowa
#będzie



KN Solvro